Rockchip Linux Memory Trouble Shooting文档

文件标识：RK-KF-YF-C36

发布版本：V1.8.0

日期：2026-03-03

文件密级：□绝密 □秘密 ■内部资料 □公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文档是处理Memory相关问题思路的汇总

产品版本

芯片名称	内核版本
ROCKCHIP 芯片	6.1/6.12

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.1.0	薛小明	2025-1-22	初始发布
V1.2.0	薛小明	2025-9-2	1. 添加/sys/kernel/debug/memblock说明 2. 添加dma_alloc_attrs返回值说明
V1.3.0	薛小明	2025-10-15	如何分析内核内存申请失败时打印的log
V1.4.0	薛小明	2025-11-13	如何获取page owner信息
V1.5.0	薛小明	2025-11-18	如何获取page pinner信息
V1.6.0	薛小明	2025-12-18	scatterlist使用注意事项
V1.7.0	薛小明	2026-01-26	cma_alloc返回失败
V1.8.0	薛小明	2026-03-03	ARM32下CMA起始地址未2M对齐导致内核启动卡死

目录

Rockchip Linux Memory Trouble Shooting文档

1. log出现swiotlb buffer is full的错误信息
2. 如何获取32bit以内的物理地址
3. kmalloc能不能指定32bit物理地址
4. 如何dump内核页表
5. dma_alloc_attrs如何执行
6. io命令访问不了物理内存
7. 内核如何预留CMA内存
8. 如何为设备预留内存
9. 如何使用dma-heap
10. 如何映射内核态内存
11. 如何映射物理内存到用户态地址
12. 内存泄漏分析
13. 冲内存分析
14. 如何获取symbol的物理地址
15. 修改系统内存
16. 内存回收策略调整
17. 内核各级页表物理地址
18. slab相关问题分析
19. 内核线性映射的cache属性
20. 如何将用户态的地址转换成物理地址
21. io读回来的数据存在一致性问题
22. ARM32环境预留奇数M的CMA引起内核奔溃
23. 用户态访问PCIe BAR空间异常
24. dts当中预留内存no-map什么意思
25. 利用/dev/mem访问物理内存
26. 内存一致性问题
27. write-combined方式映射的内存存在一致性问题
28. dma_alloc_coherent失败
29. DRM如何申请带cache的buffer
30. dma_buf fd的使用方法
31. 生成dma_buf fd的方法
32. 如何配置dma-coherent设备
33. PCIe设备如何继承PCIe控制器的dma-coherent属性
34. 如何分析CMA内存是否被系统借用
35. 设置CMA不参与系统内存分配
36. 透明大页TRANSPARENT_HUGEPAGE
37. 无法从缓存中回收内存
38. IOVA申请失败
39. /sys/kernel/debug/memblock说明
40. 关于“为什么外设预留的内存已经调用 free_reserved_area 释放后，为什么reserved节点”还会提示呢
41. 预留内存重叠
42. ARM32调试memblock卡死
43. dma_alloc_attrs 返回的 dma 地址“不在系统内存范围”原因与正确用法说明
44. 如何分析内核分配内存失败时打印的log
45. 如何获取PAGE的OWNER信息
46. 如何获取page pinner信息
47. scatterlist 使用注意事项(IOMMU/DMABUF场景)
    - 47.1 背景与基本概念
    - 47.2 常见问题与正确用法(重点：启用IOMMU + CONFIG_NEED_SG_DMA_LENGTH)
48. CMA_ALLOC返回失败
49. ARM32下CMA起始地址未2M对齐导致内核启动卡死
    - 49.1 现象描述
    - 49.2 原因分析
    - 49.3 解决方法

1. log出现swiotlb buffer is full的错误信息

默认swiotlb size为IO_TLB_DEFAULT_SIZ(64MB)，通过内核命令行传参数修改swiotlb slot个数来增大swiotlb size，比如swiotlb=0x10000，对应的swiotlb size就是(0x10000 << IO_TLB_SHIFT) >> 20 = 128MB。swiotlb单次允许申请连续的空间是IO_TLB_SEGSIZE * IO_TLB_SIZE = 256KB，当设备一次申请的buffer超过256K也会报错，当遇到该问题时可以先调整IO_TLB_SEGSIZE来确认问题，最终还是要通过调整设备驱动来解决。设备的寻址空间如果和内存地址总线位宽匹配时，就需要借助swiotlb的内存。比如设备只支持32bit即4GB的内存地址，但是设备驱动获取到的内存地址为40bit，这时候就需要借助swiotlb来进行地址转换，一般是通过dma_map_single/dma_unmap_sigle等内核接口函数实现，这个过程会自动拷贝数据以及维护cache一致性，因此会影响性能。当设备内存超过4G达到8G或者16G时，设备驱动更容易出现swiotlb buffer is full的错误，原因是很多时候系统优先从高于32位的地址申请内存，因此会导致swiotlb更容易被消耗。

2. 如何获取32bit以内的物理地址

1. 使用dts的方式预留
2. 使用GFP_DMA32参数,例如：

```
alloc_page(GFP_DMA32 | GFP_KERNEL)
__get_free_pages(GFP_DMA32,0)
```

需要注意的是dma_alloc_coherent之类的函数不支持直接设置GFP_DMA32的参数，dma_alloc_coherent在执行过程中会自动调整flag。

3. kmalloc能不能指定32bit物理地址

新的内核kmalloc不能指定GFP_DMA32来获取32bit以内的物理地址。经常看到设备驱动使用kmalloc申请内存，然后将物理地址配置给硬件，因为kmalloc的地址和硬件不匹配，导致硬件工作异常，针对这种情况可以自己创建slab来解决，例子如下:

```
struct kmem_cache *mem_cache = kmem_cache_create("test", size, 0,
SLAB_HWCACHE_ALIGN, 0);
void *p = kmem_cache_zalloc(mem_cache, GFP_DMA32|GFP_KERNEL);
```

p就是从32bit物理内存获取的。

4. 如何dump内核页表

内核打开CONFIG_PTDUMP_CORE

cat /sys/kernel/debug/kernel_page_tables便会输出内核页表信息

5. dma_alloc_attrs如何执行

dma_alloc_attrs执行比较复杂，有两条执行路径，和设备的dts是否引用iommu节点有关

1. 引用iommu

```
dma_alloc_attrs -> iommu_dma_ops.alloc -> iommu_dma_alloc
```

2. 没有引用iommu

```
dma_alloc_attrs -> dma_direct_alloc
```

需要说明的是iommu_dma_alloc申请的物理内存不一定是连续的，但是返回给设备连续的IOVA,如果有遇到内存分配相关的问题，可以根据实际情况跟踪函数

6. io命令访问不了物理内存

CONFIG_DEVMEM打开

CONFIG_STRICT_DEVMEM关闭

7. 内核如何预留CMA内存

以下两种方式

1. 配置CONFIG_CMA_SIZE_MBYTES.
2. dts添加如下：

```
reserved-memory {
    #address-cells = <2>;
    #size-cells = <2>;
    ranges;

    mem_reserved: mem-rsv@40000000 {
        reg = <0x00000000 0x40000000 0x00000000 0x00100000>;
        compatible = "shared-dma-pool";
        reusable;
        linux,cma-default;
    };
};
```

当dts配置了linux,cma-default;CONFIG_CMA_SIZE_MBYTES将失效
内核代码所在位置：kernel/dma/contiguous.c

8. 如何为设备预留内存

dts做如下修改：

```
reserved-memory {
    #address-cells = <2>;
    #size-cells = <2>;
    ranges;

    mem_reserved: mem-rsv@40000000 {
        reg = <0x00000000 0x40000000 0x00000000 0x00100000>;
        compatible = "shared-dma-pool";
        no-map;
    };
};

node: np@0 {
    memory-region = <&mem_reserved>;
};
```

内核代码所在位置: kernel/dma/coherent.c
设备驱动的probe函数添加如下补丁：

```
of_reserved_mem_device_init(dev);
```

0x40000000 - 0x40100000这1MB空间的内存空间由node设备独享，设备驱动使用dma_alloc_attrs/dma_alloc_free从这1M空间申请/释放内存。
需要注意的是不能针对这段空间做cache同步操作，原因是no-map的空间不会建立内核线性映射，而cache同步操作依赖内核线性映射。

9. 如何使用dma-heap

dma-heap依赖dma-buf，dma_buf_export + dma_buf_fd生成基于dma-buf的fd，用户态基于该fd访问内存。
以cma_heap为例:

用户态首先通过/dev/dma_heap/cma获取到dma_buf的fd，然后基于该fd访问内存,大致路径如下:

dma_buf_fops -> cma_heap_buf_ops。具体参考内核测试代码tools/testing/selftests/dmabuf-heaps/dmabuf-heap.c

10. 如何映射内核态内存

```
void *vmap(struct page **pages, unsigned int count, unsigned long flags,
pgprot_t prot)
```

其中pages存储的是需要映射的page页

11. 如何映射物理内存到用户态地址

```
int remap_pfn_range(struct vm_area_struct *vma, unsigned long addr, unsigned
long pfn, unsigned long size, pgprot_t prot)
```

remap_pfn_range一般是在mmap回调中使用，pfn物理页帧，假设物理地址是phys，pfn = phys >> PAGE_SHIFT;

可以参考内核中很多remap_pfn_range的实现方法

12. 内存泄漏分析

内核打开如下配置：

CONFIG_DEBUG_KMEMLEAK_MEM_POOL_SIZE=50000

CONFIG_DEBUG_KMEMLEAK=y

mount -t debugfs nodev /sys/kernel/debug/

1. 开始:

   echo clear > /sys/kernel/debug/kmemleak

   echo scan > /sys/kernel/debug/kmemleak

2. 运行被观察对象

3. 结束:

   cat /sys/kernel/debug/kmemleak

内核提供了测试代码,如下：

打开CONFIG_SAMPLES

打开CONFIG_DEBUG_KMEMLEAK_TEST

串口会输出堆栈信息,如下:

```
unreferenced object 0xffff8001085a3000 (size 1024):
  comm "swapper/0", pid 1, jiffies 4294892470 (age 66.712s)
  hex dump (first 32 bytes):
    00 00 00 00 01 00 00 00 08 30 5a 08 01 80 ff ff  .........0Z.....
    08 30 5a 08 01 80 ff ff 00 00 00 00 00 00 00 00  .0Z.............
  backtrace:
    [<00000000b428bb03>] slab_post_alloc_hook+0x90/0x308
    [<0000000044e70370>] __kmem_cache_alloc_node+0x15c/0x224
    [<000000004145f605>] kmalloc_trace+0x4c/0x114
    [<00000000ec1ffc0b>] kmemleak_test_init+0x80/0x2b8
    [<000000005abaeb69>] do_one_initcall+0xc4/0x2cc
    [<00000000637fb484>] do_initcall_level+0x9c/0x140
    [<00000000c6cdcb23>] do_initcalls+0x54/0x94
    [<000000006662279d>] do_basic_setup+0x1c/0x28
    [<00000000b072decb>] kernel_init_freeable+0xe0/0x158
    [<00000000fe8f4d8d>] kernel_init+0x20/0x1a4
    [<00000000d1a4f820>] ret_from_fork+0x10/0x20
```

13. 冲内存分析

内核打开如下配置：

CONFIG_KASAN=y

CONFIG_SLUB_DEBUG=y

用以下代码构造越界访问:

```
char *buf = kmalloc(64, GFP_KERNEL);
buf[64] = 'A';
```

串口会输出堆栈信息,如下:

```
[    7.280718][    T1]
==================================================================
[    7.280742][    T1] BUG: KASAN: slab-out-of-bounds in kasan_test+0x3c/0x60
[    7.280773][    T1] Write of size 1 at addr ffff80010d02f540 by task
swapper/0/1
[    7.280794][    T1]
[    7.280809][    T1] CPU: 4 PID: 1 Comm: swapper/0 Not tainted 6.1.115 #389
[    7.280832][    T1] Hardware name: Rockchip RK3588 EVB1 LP4 V10 Board (DT)
[    7.280849][    T1] Call trace:
[    7.280863][    T1]  dump_backtrace+0xf4/0x114
[    7.280890][    T1]  show_stack+0x18/0x24
[    7.280913][    T1]  dump_stack_lvl+0x6c/0x90
[    7.280935][    T1]  print_report+0x15c/0x4a0
[    7.280957][    T1]  kasan_report+0xac/0x100
[    7.280977][    T1]  __asan_store1+0x6c/0x70
[    7.280997][    T1]  kasan_test+0x3c/0x60
[    7.281017][    T1]  do_one_initcall+0xe4/0x3d4
[    7.281038][    T1]  do_initcall_level+0xd4/0x1e8
[    7.281061][    T1]  do_initcalls+0x54/0x98
[    7.281082][    T1]  do_basic_setup+0x64/0x78
[    7.281103][    T1]  kernel_init_freeable+0x144/0x1c0
[    7.281125][    T1]  kernel_init+0x20/0x1b4
[    7.281146][    T1]  ret_from_fork+0x10/0x20
[    7.281167][    T1]
[    7.281180][    T1] Allocated by task 1:
[    7.281196][    T1]  kasan_set_track+0x4c/0x80
[    7.281216][    T1]  kasan_save_alloc_info+0x24/0x30
[    7.281238][    T1]  __kasan_kmalloc+0xa0/0xb8
[    7.281258][    T1]  kmalloc_trace+0x68/0x120
[    7.281280][    T1]  kasan_test+0x30/0x60
[    7.281300][    T1]  do_one_initcall+0xe4/0x3d4
[    7.281320][    T1]  do_initcall_level+0xd4/0x1e8
[    7.281341][    T1]  do_initcalls+0x54/0x98
[    7.281362][    T1]  do_basic_setup+0x64/0x78
[    7.281383][    T1]  kernel_init_freeable+0x144/0x1c0
[    7.281405][    T1]  kernel_init+0x20/0x1b4
[    7.281425][    T1]  ret_from_fork+0x10/0x20
[    7.281445][    T1]
```

```
[    7.281458][    T1] The buggy address belongs to the object at
ffff80010d02f500
[    7.281458][    T1]  which belongs to the cache kmalloc-64 of size 64
[    7.281479][    T1] The buggy address is located 0 bytes to the right of
[    7.281479][    T1]  64-byte region [ffff80010d02f500, ffff80010d02f540)
[    7.281502][    T1]
[    7.281515][    T1] The buggy address belongs to the physical page:
[    7.281530][    T1] page:000000001d71e131 refcount:1 mapcount:0
mapping:0000000000000000 index:0x0 pfn:0x4340b
[    7.281553][    T1] flags: 0x4000000000000200(slab|zone=1)
[    7.281579][    T1] raw: 4000000000000200 0000000000000000 dead000000000122
ffff800100004280
[    7.281602][    T1] raw: 0000000000000000 0000000080800080 00000001ffffffff
0000000000000000
[    7.281619][    T1] page dumped because: kasan: bad access detected
[    7.281635][    T1]
[    7.281647][    T1] Memory state around the buggy address:
[    7.281665][    T1]  ffff80010d02f400: fa fb fb fb fb fb fb fb fc fc fc fc fc
fc fc fc
[    7.281683][    T1]  ffff80010d02f480: fa fb fb fb fb fb fb fb fc fc fc fc fc
fc fc fc
[    7.281702][    T1] >ffff80010d02f500: 00 00 00 00 00 00 00 00 fc fc fc fc fc
fc fc fc
[    7.281718][    T1]                                                 ^
[    7.281736][    T1]  ffff80010d02f580: fc fc fc fc fc fc fc fc fc fc fc fc fc
fc fc fc
[    7.281754][    T1]  ffff80010d02f600: fc fc fc fc fc fc fc fc fc fc fc fc fc
fc fc fc
[    7.281771][    T1]
==================================================================
```

14. 如何获取symbol的物理地址

symbol指函数,全局变量等。

假设函数名是function，function对应的物理地址是__pa_symbol(function);

假设全局变量名是val，val对应的物理地址是__pa_symbol(val);

这个可以作为分析问题的辅助手段。

15. 修改系统内存

有时候为了分析问题需要限制系统内存，可以简单通过修改内核memblock_add函数实现。因为内核伙伴系统的初始化依赖memblock的信息。

memblock_add函数原型如下:

```
int __init_memblock memblock_add(phys_addr_t base, phys_addr_t size)
{
    phys_addr_t end = base + size - 1;
    memblock_dbg("%s: [%pa-%pa] %pS\n", __func__,
                 &base, &end, (void *)_RET_IP_);
    return memblock_add_range(&memblock.memory, base, size, MAX_NUMNODES, 0);
}
```

可以通过判断base和size做针对性的调整，类似如下补丁:

```
if (base == 0x0000000100000000)
    return 0;
if (base == 0x0000000009400000)
    size = 0xa6c00000;
```

上面的补丁功能包括:

1. 忽略0x100000000开始的地址空间,因此这片地址空间不会加入系统内存。
2. 修改0x9400000开始的地址空间的size。

16. 内存回收策略调整

有时候为了分析性能和内存异常的问题，需要调整内存交换的策略，主要是对/proc/sys/vm路径下节点的操作。

1. swappiness

   swappiness值越大，系统更倾向使用swap分区交换内存，越小，更倾向于使用内存保存数据。

2. dirty_ratio和dirty_background_ratio

   控制脏页和总内存的比重,如果超过dirty_ratio，内核主动将脏页写入磁盘，如果超过dirty_background_ratio,后台进程会开始将脏页写入磁盘。

3. min_free_kbytes

   指定系统保留的最小空闲内存量，以防止内存完全耗尽，影响系统运行。

   有时候为了测试系统稳定性，可以将min_free_kbytes调高，让系统尽早并且频繁的执行回收内存的动作。

4. vfs_cache_pressure

   表示内核回收directory和inode cache内存的倾向。

   等于100，保持directory和inode cache内存和page cache之间的平衡。

   大于100，更倾向于回收directory和inode cache，将更多的内存用于page cache。

   小于100，更倾向于保留directory和inode cache，从而减少目录和inode信息的读取次数。

   大多数时候这个值不需要测试,除非碰到虚拟文件系统相关性能时候,才需要调整这个参数。

17. 内核各级页表物理地址

以3级页表为例:

第一级页表物理地址 = __pa_symbol(swapper_pg_dir);

第二级页表物理地址 = __pa_symbol(bm_pmd);

第三级页表物理地址 = __pa_symbol(bm_pte);

以上地址用于辅助分析相关问题。

18. slab相关问题分析

1. 内核打开如下配置：

   CONFIG_SLUB_DEBUG=y

   CONFIG_SLUB_DEBUG_ON=y

2. cat /proc/slabinfo

   统计slub变化情况,假设kmalloc-128异常。

3. 开始trace: `echo 1 > /sys/kernel/slab/kmalloc-128/trace`

   执行测试程序。

   停止trace: `echo 0 > /sys/kernel/slab/kmalloc-128/trace`

   保存trace数据到文件:

   ```
   cat /proc/kmsg > slab-trace-128.log

   grep "TRACE kmalloc-128 free" slab-trace-128.log | awk '{print $7}' | sort > free.txt

   grep "TRACE kmalloc-128 alloc" slab-trace-128.log | awk '{print $7}' | sort > alloc.txt
   ```

   就能统计kmalloc-128相关的分配/释放信息。

19. 内核线性映射的cache属性

内核建立线性映射的函数执行路径大致如下:

```
paging_init -> map_mem -> __map_memblock(pgdp, start, end,
pgprot_tagged(PAGE_KERNEL), flags) -> __create_pgd_mapping -> alloc_init_pud -
>init_pmd -> init_pte
```

可以看到使用的prot参数是pgprot_tagged(PAGE_KERNEL),即带MTE(memory tag extension)的cache，所以对kmalloc或者phys_to_virt返回的地址进行读写操作都是带cache属性的。

需要说明的是，对于ARM64来说，内核线性映射的建立受CONFIG_RODATA_FULL_DEFAULT_ENABLED配置影响，这个配置关闭的情况下，内核会按照block的方式映射，这个带来的好处是映射的时间缩短，内核页表占据的空间缩小。但是CONFIG_RODATA_FULL_DEFAULT_ENABLED提供了重要的安全保护机制，通过防止对只读数据的修改来增强内核的安全性，默认的CONFIG_RODATA_FULL_DEFAULT_ENABLED是打开的。

20. 如何将用户态的地址转换成物理地址

主要是利用get_user_pages函数，内核有完整的测试代码，如下:

内核代码：mm/gup_test.c

打开内核CONFIG_GUP_TEST编译，生成/sys/kernel/debug/gup_test节点

用户态测试代码：tools/testing/selftests/vm/gup_test.c

进入文件所在目录：make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- LDFLAGS="-static"编译生成gup_test工具，使用gup_test工具测试。

21. io读回来的数据存在一致性问题

io命令操作的是/dev/mem节点，这个节点默认的映射方式是write_combined，可以简单做如下补丁确保io正常:

修改drivers/char/mem.c当中的mmap_mem函数

```
vma->vm_page_prot = pgprot_noncached(vma->vm_page_prot);
```

以上补丁将映射方式修改成device_memory，既没有cache也没有write buffer。

22. ARM32环境预留奇数M的CMA引起内核奔溃

ARM32要求页表按照2M对齐，因此预留CMA空间的时候需要规避这个问题。

23. 用户态访问PCIe BAR空间异常

用户态代码有可能受编译器的优化影响，将多个访问BAR空间的操作优化成一个操作，从而导致地址不对齐。

目前针对这种情况只能根据实际情况调整用户代码，在相应的访问BAR空间的地方加内存屏障dmb/dsb防止编译器优化。需要说明的是地址对齐是指地址是访问size的整数倍，比如地址是0xffff0008，size是2/4/8字节。

24. dts当中预留内存no-map什么意思

内核不会对no-map区域的物理内存建立线性映射，意味着要手动为这片内存建立映射，并且不能对这片地址做cache操作。

25. 利用/dev/mem访问物理内存

有时候应用程序不想为了访问一个物理内存专门写一个misc驱动，可以利用/dev/mem节点访问，代码片段如下:

CONFIG_DEVMEM打开

CONFIG_STRICT_DEVMEM关闭

```
unsigned long phys_addr = 0x80000000;

unsigned long length = 0x100000;

int fd = open("/dev/mem", O_RDWR);

void *addr = mmap(NULL, length, PROT_READ|PROT_WRITE, MAP_SHARED, fd,
phys_addr);

memset(addr, 0x34, length);

munmap(addr, length);
```

以上代码将物理地址0x80000000 - 0x80100000 1M大小的物理内存写成0x34

26. 内存一致性问题

1. 将映射修改成non-cache验证是否一致性问题。
2. 如果是一致性问题，内核使用dma_sync_single_for_cpu/dma_sync_single_for_device维护cache一致性。

   CPU访问内存之前调用dma_sync_single_for_cpu(invalid cache)。

   设备访问内存之前调用dma_sync_single_for_device(clean cache)。

   需要注意的是dma_sync_single_for_cpu/dma_sync_single_for_device接受的是物理地址。

27. write-combined方式映射的内存存在一致性问题

虽然write-combined映射的内存没有被cache，但是对其写操作有可能被缓存，因此需要使用内存屏障操作dmb/dsb确保数据写到物理内存。

28. dma_alloc_coherent失败

GFP_ATOMIC会导致分配过程走进atomic pool，如果atomic pool配置较小，会引起分配失败，如果遇到这种问题有以下两种尝试:

1. 把GFP_ATOMIC改成GFP_KERNEL。
2. 调整atomic pool大小，内核命令行添加：coherent_pool=4M，设置atomic pool为4MB。

29. DRM如何申请带cache的buffer

用户态代码drm_rockchip_gem_create.flags或者drm_mode_create_dumb.flags需要带上ROCKCHIP_BO_CACHABLE。

30. dma_buf fd的使用方法

主要三步，如下:

```
1. struct dma_buf *buf = dma_buf_get(fd);
2. struct dma_buf_attachment *attach = dma_buf_attach(buf, dev);
3. struct sg_table *table = dma_buf_map_attachment(attach, DMA_BIDIRECTIONAL);
```

31. 生成dma_buf fd的方法

主要有两步,如下:

```
1. struct dma_buf *buf = dma_buf_export(struct dma_buf_export_info *info);
2. dma_buf_fd(buf, O_CLOEXEC);
```

32. 如何配置dma-coherent设备

在dts定义设备节点的时候加入一行dma-coherent；即表示是dma-coherent设备。

dma-coherent意味着设备能够直接访问CPU的cache，不需要软件执行额外的刷cache操作，前提是SoC支持设备dma一致性的访问。

platform设备dma-coherent初始化的路径大致如下:

```
platform_dma_configure -> of_dma_configure -> of_dma_is_coherent ->
arch_setup_dma_ops -> dev->dma_coherent = coherent;
```

当设备调用dma_alloc_coherent申请内存时，cpu的页表属性会从默认的pgprot_writecombine变更为normal，即从non-cached变更为cached。

当设备调用dma_sync_single_for_cpu/dma_sync_single_for_device等接口做cache维护操作时，会先调用dev_is_dma_coherent判断设备是否dma-coherent，如果是则跳过维护cache的操作，因此对于驱动来说是无感的。

33. PCIe设备如何继承PCIe控制器的dma-coherent属性

发生在PCIe设备枚举阶段,以RK芯片为例:

```
rk_add_pcie_port -> dw_pcie_host_init -> pci_host_probe ->
pci_scan_root_bus_bridge -> pci_scan_child_bus ->pci_scan_child_bus_extend ->
pci_scan_slot -> pci_scan_single_device -> pci_scan_device-> pci_device_add ->
device_add ->bus_probe_device -> device_initial_probe -> __device_attach ->
__device_attach_driver -> driver_probe_device ->
__driver_probe_device -> really_probe -> dev->bus->dma_configure ->
pci_dma_configure -> of_dma_configure ->of_dma_is_coherent -> arch_setup_dma_ops
-> dev->dma_coherent = coherent;
```

需要注意的是pci_dma_configure这里使用bridge->parent->of_node作为参数调用of_dma_configure，bridge->parent->of_node就是PCIe控制器节点，因此PCIe设备继承了PCIe控制器的dma-coherent的属性。

34. 如何分析CMA内存是否被系统借用

可以借助内核trace功能,操作步骤如下:

```
echo 1 > /sys/kernel/debug/tracing/events/kmem/mm_page_alloc_zone_locked/enable

echo 1 > /sys/kernel/debug/tracing/tracing_on
```

以上两步打开mm_page_alloc_zone_locked的trace，mm_page_alloc_zone_locked主要发生在系统从CMA借用page阶段。

重复执行以下命令,也可以写成脚本执行:

```
cat /sys/kernel/debug/tracing/trace | grep migratetype=3
```

如果有借用,会出现类似下面的log:

```
ActivityManager-743     [002] d..3.   315.474016: mm_page_alloc_zone_locked:
page=00000000734f4659 pfn=0x434d6b order=0 migratetype=3 percpu_refill=1
ActivityManager-743     [002] d..3.   315.474017: mm_page_alloc_zone_locked:
page=00000000734f4659 pfn=0x434d6b order=0 migratetype=3 percpu_refill=1
ActivityManager-743     [002] d..3.   315.474018: mm_page_alloc_zone_locked:
page=000000003cebbd4d pfn=0x43378a order=0 migratetype=3 percpu_refill=1
ActivityManager-743     [002] d..3.   315.474019: mm_page_alloc_zone_locked:
page=000000003cebbd4d pfn=0x43378a order=0 migratetype=3 percpu_refill=1
ActivityManager-743     [002] d..3.   315.474021: mm_page_alloc_zone_locked:
page=000000006816a4cc pfn=0x435545 order=0 migratetype=3 percpu_refill=1
ActivityManager-743     [002] d..3.   315.474021: mm_page_alloc_zone_locked:
page=000000006816a4cc pfn=0x435545 order=0 migratetype=3 percpu_refill=1
ActivityManager-743     [002] d..3.   315.474023: mm_page_alloc_zone_locked:
page=00000000a1971528 pfn=0x435547 order=0 migratetype=3 percpu_refill=1
```

CMA的所属的内存物理地址范围是[0x430000000 - 0x440000000]，可以看到进程ActivityManager从CMA的区域借用page，每次借用一个page，migratetype=3表示CMA的page链表

35. 设置CMA不参与系统内存分配

内核CONFIG_CMA_INACTIVE打开。

36. 透明大页TRANSPARENT_HUGEPAGE

内核打开CONFIG_TRANSPARENT_HUGEPAGE。

在内存富裕环境下，透明大页有助于降低CPU负载，可能带来的问题是:

1. 内存分配延迟
2. 性能可能波动
3. 可能引起内存浪费
4. 可能存在的兼容性问题

用户可通过如下节点开关透明大页功能：

cat /sys/kernel/mm/transparent_hugepage/enabled

always [madvise] never

echo always > /sys/kernel/mm/transparent_hugepage/enabled

37. 无法从缓存中回收内存

cat /proc/meminfo发现Reclaimable还有很多内存，但是内核的分配却失败了，这个大概率是因为申请的表示设置为GFP_ATOMIC，阻止了分配进入内存回收路径，可以尝试修改成GFP_ATOMIC，允许内核分配路径进入回收路径。

38. IOVA申请失败

如果npu驱动log提示以下信息：

```
failed to allocate IOVA: -12
```

修改以下代码确认是否是因为size对齐引起的IOVA不足:

```
drivers/iommu/iova.c当中的__alloc_and_insert_iova_range
```

将参数size_aligned强制设置为false，然后再测试

如果以上补丁没有规避问题，可能是IOVA空间确实不满足应用场景，需要从整体减小对IOVA的使用

39. /sys/kernel/debug/memblock说明

/sys/kernel/debug/memblock/目录下有两个节点，分别是

```
/sys/kernel/debug/memblock/memory
```

```
/sys/kernel/debug/memblock/reserved
```

memory节点表示系统识别的所有内存
reserved节点表示系统预留的所有内存

其中reserved是memory的子集

分别打开得到如下信息：

```
cat /sys/kernel/debug/memblock/memory
   0: 0x0000000000200000..0x00000000083fffff
   1: 0x0000000009400000..0x00000000effffffe
```

```
cat /sys/kernel/debug/memblock/reserved
   0: 0x0000000000110000..0x00000000001effff
   1: 0x0000000000410000..0x00000000030fcfff
   2: 0x0000000008300000..0x000000000834c07f
   3: 0x0000000010000000..0x00000000107fffff
   4: 0x0000000040000000..0x0000000047ffffff
   5: 0x00000000e8000000..0x00000000ebbfffff
   6: 0x00000000ebd4a000..0x00000000ebd59fff
   7: 0x00000000eefab2c0..0x00000000ef5ab7c7
   8: 0x00000000ef5ab800..0x00000000ef5abc8f
   9: 0x00000000ef5abcc0..0x00000000ef5abe47
  10: 0x00000000ef5abe80..0x00000000ef693fff
  11: 0x00000000ef696000..0x00000000ef69705f
  12: 0x00000000ef697080..0x00000000ef69710f
  13: 0x00000000ef697140..0x00000000ef69725f
  14: 0x00000000ef697280..0x00000000ef6972df
  15: 0x00000000ef697300..0x00000000ef697307
  16: 0x00000000ef697340..0x00000000ef697740
  17: 0x00000000ef697780..0x00000000ef697b80
  18: 0x00000000ef697bc0..0x00000000ef697f0f
  19: 0x00000000ef697f40..0x00000000ef6a8f47
  20: 0x00000000ef6a8f78..0x00000000ef6a8fa5
  21: 0x00000000ef6a8fa8..0x00000000ef6a8fd5
  22: 0x00000000ef6a8fd8..0x00000000ef6a9006
  23: 0x00000000ef6a9008..0x00000000ef6a9034
  24: 0x00000000ef6a9038..0x00000000ef6a9064
  25: 0x00000000ef6a9068..0x00000000ef6a9099
  26: 0x00000000ef6a90a0..0x00000000ef6a90d1
  27: 0x00000000ef6a90d8..0x00000000ef6a9105
  28: 0x00000000ef6a9108..0x00000000ef6a9135
  29: 0x00000000ef6a9138..0x00000000ef6a9168
  30: 0x00000000ef6a9170..0x00000000ef6a919b
  31: 0x00000000ef6a91a0..0x00000000ef6a91cb
  32: 0x00000000ef6a91d0..0x00000000ef6a91fa
  33: 0x00000000ef6a9200..0x00000000ef6a922a
```

```
34: 0x00000000ef6a9230..0x00000000ef6a925c
35: 0x00000000ef6a9260..0x00000000ef6a928c
36: 0x00000000ef6a9290..0x00000000ef6a92bb
37: 0x00000000ef6a92c0..0x00000000ef6a92eb
38: 0x00000000ef6a92f0..0x00000000ef6a931b
39: 0x00000000ef6a9320..0x00000000ef6a934b
40: 0x00000000ef6a9350..0x00000000ef6a937b
41: 0x00000000ef6a9380..0x00000000ef6a93ab
42: 0x00000000ef6a93b0..0x00000000ef6a93de
43: 0x00000000ef6a93e0..0x00000000ef6a940e
44: 0x00000000ef6a9410..0x00000000ef6a943e
45: 0x00000000ef6a9440..0x00000000ef6a946e
46: 0x00000000ef6a9470..0x00000000ef6a949e
47: 0x00000000ef6a94a0..0x00000000ef6a94ce
48: 0x00000000ef6a94d0..0x00000000ef6a94fe
49: 0x00000000ef6a9500..0x00000000ef6a952e
50: 0x00000000ef6a9530..0x00000000ef6a955e
51: 0x00000000ef6a9560..0x00000000ef6a958e
52: 0x00000000ef6a9590..0x00000000ef6a95bd
53: 0x00000000ef6a95c0..0x00000000ef6a95ed
54: 0x00000000ef6a95f0..0x00000000ef6a961d
55: 0x00000000ef6a9620..0x00000000ef6a964d
56: 0x00000000ef6a9650..0x00000000ef6a967e
57: 0x00000000ef6a9680..0x00000000ef6a96ae
58: 0x00000000ef6a96b0..0x00000000ef6a96de
59: 0x00000000ef6a96e0..0x00000000ef6a970e
60: 0x00000000ef6a9710..0x00000000ef6a988b
61: 0x00000000ef6a9890..0x00000000ef6a98bb
62: 0x00000000ef6a98c0..0x00000000ef6a98eb
63: 0x00000000ef6a98f0..0x00000000ef6a991b
64: 0x00000000ef6a9920..0x00000000ef6a994b
65: 0x00000000ef6a9950..0x00000000ef6a997b
66: 0x00000000ef6a9980..0x00000000ef6a99ab
67: 0x00000000ef6a99b0..0x00000000ef6a99db
68: 0x00000000ef6a99e0..0x00000000ef6a9a0b
69: 0x00000000ef6a9a10..0x00000000ef6a9a3c
70: 0x00000000ef6a9a40..0x00000000ef6a9a6c
71: 0x00000000ef6a9a70..0x00000000ef6a9a9c
72: 0x00000000ef6a9aa0..0x00000000ef6a9acc
73: 0x00000000ef6a9ad0..0x00000000ef6a9afc
74: 0x00000000ef6a9b00..0x00000000ef6a9b30
75: 0x00000000ef6a9b38..0x00000000ef6a9b63
76: 0x00000000ef6a9b68..0x00000000ef6a9b93
77: 0x00000000ef6a9b98..0x00000000ef6a9bc8
78: 0x00000000ef6a9bd0..0x00000000ef6a9c00
79: 0x00000000ef6a9c08..0x00000000ef6a9c38
80: 0x00000000ef6a9c40..0x00000000ef6a9c70
81: 0x00000000ef6a9c78..0x00000000ef6a9ca8
82: 0x00000000ef6a9cb0..0x00000000ef6a9ce0
83: 0x00000000ef6a9ce8..0x00000000ef6a9d19
84: 0x00000000ef6a9d20..0x00000000ef6a9d51
85: 0x00000000ef6a9d58..0x00000000effffff
```

在内核的命令行添加memblock=debug，内核在开机阶段会把所有的预留内存信息打印出来，截取部分，类似如下：

```
[    1.041012][    T0] memblock_remove: [0x0001000000000000-0x0000fffffffffffe]
arm64_memblock_init+0x28/0x268
[    1.041908][    T0] memblock_remove: [0x0000004000000000-0x0000003fffffffe]
arm64_memblock_init+0x80/0x268
[    1.042796][    T0] memblock_add: [0x000000000a200000-0x000000000a364fff]
arm64_memblock_init+0x14c/0x268
[    1.043670][    T0] memblock_reserve: [0x000000000a200000-0x000000000a364fff]
arm64_memblock_init+0x164/0x268
[    1.044575][    T0] memblock_reserve: [0x0000000000410000-0x00000000030fffff]
arm64_memblock_init+0x1e8/0x268
[    1.045476][    T0] memblock_reserve: [0x0000000008300000-0x000000000834bfff]
early_init_fdt_scan_reserved_mem+0x7c/0xac
[    1.046470][    T0] memblock_reserve: [0x000000000a200000-0x000000000a3642f3]
early_init_fdt_scan_reserved_mem+0x7c/0xac
[    1.050206][    T0] memblock_reserve: [0x0000000010000000-0x00000000107fffff]
early_init_dt_reserve_memory+0x58/0x78
[    1.051178][    T0] memblock_reserve: [0x00000000edf00000-0x00000000ee059fff]
early_init_dt_reserve_memory+0x58/0x78
[    1.053245][    T0] memblock_reserve: [0x00000000ebd4a000-0x00000000ebd59fff]
early_init_dt_reserve_memory+0x58/0x78
[    1.054221][    T0] memblock_reserve: [0x0000000000110000-0x00000000001effff]
early_init_dt_reserve_memory+0x58/0x78
[    1.059830][    T0] memblock_phys_alloc_range: 4096 bytes align=0x1000
from=0x0000000000000000 max_addr=0x0000000000000001
early_pgtable_alloc+0x24/0xdc
[    1.061073][    T0] memblock_reserve: [0x00000000effff000-0x00000000effffffe]
memblock_alloc_range_nid+0xd0/0x1a4
[    1.062006][    T0] memblock_phys_alloc_range: 4096 bytes align=0x1000
from=0x0000000000000000 max_addr=0x0000000000000001
early_pgtable_alloc+0x24/0xdc
[    1.063245][    T0] memblock_reserve: [0x00000000efffe000-0x00000000efffefff]
memblock_alloc_range_nid+0xd0/0x1a4
[    1.064198][    T0] memblock_phys_alloc_range: 4096 bytes align=0x1000
from=0x0000000000000000 max_addr=0x0000000000000001
early_pgtable_alloc+0x24/0xdc
[    1.065435][    T0] memblock_reserve: [0x00000000efffd000-0x00000000efffdfff]
memblock_alloc_range_nid+0xd0/0x1a4
[    1.066387][    T0] memblock_phys_alloc_range: 4096 bytes align=0x1000
from=0x0000000000000000 max_addr=0x0000000000000001
early_pgtable_alloc+0x24/0xdc
[    1.067629][    T0] memblock_reserve: [0x00000000efffc000-0x00000000efffcfff]
memblock_alloc_range_nid+0xd0/0x1a4
[    1.068586][    T0] memblock_phys_alloc_range: 4096 bytes align=0x1000
from=0x0000000000000000 max_addr=0x0000000000000001
early_pgtable_alloc+0x24/0xdc
[    1.069832][    T0] memblock_reserve: [0x00000000efffb000-0x00000000efffbfff]
memblock_alloc_range_nid+0xd0/0x1a4
[    1.070784][    T0] memblock_phys_alloc_range: 4096 bytes align=0x1000
from=0x0000000000000000 max_addr=0x0000000000000001
early_pgtable_alloc+0x24/0xdc
[    1.072028][    T0] memblock_reserve: [0x00000000efffa000-0x00000000efffafff]
memblock_alloc_range_nid+0xd0/0x1a4
[    1.072973][    T0] memblock_phys_alloc_range: 524288 bytes align=0x1000
from=0x0000000000000000 max_addr=0x0000000000000000
arm64_kfence_alloc_pool+0x30/0x74
[    1.074258][    T0] memblock_reserve: [0x00000000eff7a000-0x00000000efff9fff]
memblock_alloc_range_nid+0xd0/0x1a4
```

```
[    5.421163][    T0] memblock_alloc_try_nid: 45 bytes align=0x8 nid=-1
from=0x0000000000000000 max_addr=0x0000000000000000
early_init_dt_alloc_memory_arch+0x28/0x58
[    5.422491][    T0] memblock_reserve: [0x00000000ef6a9008-0x00000000ef6a9034]
memblock_alloc_range_nid+0xd0/0x1a4
[    5.423456][    T0] memblock_alloc_try_nid: 47 bytes align=0x8 nid=-1
from=0x0000000000000000 max_addr=0x0000000000000000
early_init_dt_alloc_memory_arch+0x28/0x58
[    5.424790][    T0] memblock_reserve: [0x00000000ef6a8fd8-0x00000000ef6a9006]
memblock_alloc_range_nid+0xd0/0x1a4
[    5.425753][    T0] memblock_alloc_try_nid: 46 bytes align=0x8 nid=-1
from=0x0000000000000000 max_addr=0x0000000000000000
early_init_dt_alloc_memory_arch+0x28/0x58
[    5.427088][    T0] memblock_reserve: [0x00000000ef6a8fa8-0x00000000ef6a8fd5]
memblock_alloc_range_nid+0xd0/0x1a4
[    5.428057][    T0] memblock_alloc_try_nid: 46 bytes align=0x8 nid=-1
from=0x0000000000000000 max_addr=0x0000000000000000
early_init_dt_alloc_memory_arch+0x28/0x58
[    5.429389][    T0] memblock_reserve: [0x00000000ef6a8f78-0x00000000ef6a8fa5]
memblock_alloc_range_nid+0xd0/0x1a4
[    5.568705][    T0] memblock_alloc_try_nid: 8 bytes align=0x40 nid=-1
from=0x0000000000000000 max_addr=0x0000000000000000
pcpu_setup_first_chunk+0xb0/0x950
[    5.569977][    T0] memblock_reserve: [0x00000000ef6a8f40-0x00000000ef6a8f47]
memblock_alloc_range_nid+0xd0/0x1a4
[    5.570910][    T0] memblock_alloc_try_nid: 8 bytes align=0x40 nid=-1
from=0x0000000000000000 max_addr=0x0000000000000000
pcpu_setup_first_chunk+0xd8/0x950
[    5.572178][    T0] memblock_reserve: [0x00000000ef697300-0x00000000ef697307]
memblock_alloc_range_nid+0xd0/0x1a4
[    5.573111][    T0] memblock_alloc_try_nid: 32 bytes align=0x40 nid=-1
from=0x0000000000000000 max_addr=0x0000000000000000
pcpu_setup_first_chunk+0x104/0x950
[    5.574396][    T0] memblock_reserve: [0x00000000ef6972c0-0x00000000ef6972df]
memblock_alloc_range_nid+0xd0/0x1a4
[    5.575332][    T0] memblock_alloc_try_nid: 64 bytes align=0x40 nid=-1
from=0x0000000000000000 max_addr=0x0000000000000000
pcpu_setup_first_chunk+0x12c/0x950
[    5.576624][    T0] memblock_reserve: [0x00000000ef697280-0x00000000ef6972bf]
memblock_alloc_range_nid+0xd0/0x1a4
```

内核除了显式通过 CMA 预留的内存之外，还会有一批“永久”占用或长期占用的物理内存区域。例如：

1. 线性映射所对应的多级页表
2. struct page 元数据（页框描述符）
3. percpu 变量
4. 内核代码段与只读数据、初始化数据等
5. 设备树（DTB）
6. 各类内核分配的全局哈希表（如文件系统 dentry / inode / buffer 相关结构的初始或常驻部分）
7. 早期启动阶段临时或保留的 memblock 区域（尚未释放或尚未从 memblock 结构中移除）

以 struct page 占用的物理内存为例：

假设系统物理内存为 4 GB，PAGE_SIZE 为 4 KB，则物理页数为： 4 GB / 4 KB = 1024 × 1024（即 2^20 个页）。

在 ARM64 环境下，struct page 的大小通常为 64 字节（具体大小可能因配置而异，这里以 64B 估算），因此所有页描述符占用的内存为：64 × 1024 × 1024 = 64 MB。

这部分属于固定开销，无法直接供普通页分配器分配给用户态或一般内核动态内存使用。

40. 关于“为什么外设预留的内存已经调用 free_reserved_area 释放后，为什么reserved节点”还会提示呢

free_reserved_area() 的作用是把那段内存加入伙伴系统（page allocator），使其可以被再次分配使用；

但是 memblock 子系统内部仍然保存着这段区域的 “reserved” 记录，若确实需要同步移除 memblock 中的保留标记，还需要调用 memblock_free()（或相关接口）将该区域从 memblock.reserved 列表中删除。

41. 预留内存重叠

当两个（或多个）预留的物理内存区间发生重叠时，内核早期的 memblock 子系统在执行 memblock_reserve（或解析 DTS 中 reserved-memory 节点）时，会检测到 overlap，并输出相应的告警信息。

随后 memblock 会把这些有交集的区间合并成一个更大的连续保留区域。在reserved节点就会看到合并后的“大块”区域。表面上“预留成功”，但本来希望隔离开的两个外设各自使用的内存实际上在物理上重叠，极易导致冲内存现象发生引起的数据异常。所以在实际编写代码过程中应该禁止出现预留内存重叠，预留内存重叠例子如下：

```
/ {
    #address-cells = <2>;
    #size-cells = <2>;

    reserved-memory {
        #address-cells = <2>;
        #size-cells = <2>;
        ranges;

        /* 设备 A carveout: 2MB @ 0x8000_0000 */
        mem_devA: devA@80000000 {
            compatible = "shared-dma-pool";
            reg = <0x00000000 0x80000000  0x00000000 0x00200000>; /* size =
0x200000 */
            no-map;
        };

        /* 设备 B carveout: 0x110000 (1,118,208 bytes) @ 0x8010_0000，与 A 部分重叠
*/
        mem_devB: devB@80100000 {
            compatible = "shared-dma-pool";
            reg = <0x00000000 0x80100000  0x00000000 0x00110000>;
            no-map;
        };
    };
};
```

42. ARM32调试memblock卡死

在 ARM32 平台上，启动参数加入 memblock=debug 后，系统早期启动阶段可能出现“卡死”/无进一步输出的现象。根本原因在于：

1. earlycon通过 fixmap 建立一段临时的寄存器虚拟映射，用于在内核尚未完成完整内存管理初始化前输出日志。
2. 启动流程推进到一定阶段时，内核会撤销或重建这类临时 fixmap 映射。
3. 启用 memblock=debug 后，memblock 子系统在解析各内存区间时会打印大量调试信息，显著增加对早期串口寄存器的访问频率，且这些打印可能恰好跨越 fixmap 被撤销或尚未重新建立的窗口。
4. 一旦日志路径继续尝试访问已经失效的寄存器虚拟地址，就会导致系统卡死现象。

memblock=debug打开之后，可以先删除earlycon来绕过这个问题。

43. dma_alloc_attrs 返回的 dma 地址“不在系统内存范围”原因与正确用法说明

1. 现象

   驱动里调用：

   ```
   cpu_addr = dma_alloc_attrs(dev, size, &dma_handle, GFP_KERNEL, attrs);
   ```

   打印发现：

   - cpu_addr（内核虚拟地址）正常，可读写。
   - dma_handle却是一个看似超出本机DRAM物理地址范围的数值。

     实际上这是正常的 DMA 总线地址（Bus/DMA Address），并非CPU物理地址。观察内核代码可以发现，cpu_addr是用真实的CPU物理地址做的页表映射。所以CPU可以正常访问该内核虚拟地址。

2. 核心原因概述

   dma_handle代表设备应使用的 DMA/Bus 地址。

   在存在address translation（地址平移窗口）的总线（PCIe / 某些片上内部总线）上：

   设备DMA地址 != CPU物理地址。

   转换规则由设备树(DTS)或ACPI提供的dma-ranges描述，设别枚举时解析生成dev->dma_range_map。

   之后所有“物理地址 → 设备DMA地址”的换算都会走translate_phys_to_dma()。

   所有“设备DMA地址 → 物理地址”的换算都会走translate_dma_to_phys()。

   因此dma_handle可能落在一个“总线地址窗口”，该数值本身在系统物理地址空间中没有直接意义。

3. 关键代码

   当dev->dma_range_map存在时，dma_alloc_attrs会调用translate_phys_to_dma，如下：

```
static inline dma_addr_t translate_phys_to_dma(struct device *dev,
        phys_addr_t paddr)
{
    const struct bus_dma_region *m;

    for (m = dev->dma_range_map; m->size; m++)
        if (paddr >= m->cpu_start && paddr - m->cpu_start < m->size)
            return (dma_addr_t)paddr - m->offset;

    /* make sure dma_capable fails when no translation is available */
    return DMA_MAPPING_ERROR;
}
```

核心公式：
dma = paddr - offset
而offset = cpu_start - dma_start（生成 dev->dma_range_map 时就已计算）。
dma = (paddr - cpu_start) + dma_start（即“区间内偏移 + 设备侧基准”）。

4. DTS 里的dma-ranges片段

```
dma-ranges = <0x42000000 0 0x80000000  0 0x40000000  0 0x40000000>;
```

- 第1个cell 0x42000000：标志 + 空间类型（PCI prefetchable memory space，非真正地址位）。
- 接下来的两个cell (0, 0x80000000)：设备侧（PCI总线）起始地址dma_start = 0x00000000_80000000。
- 父级两个cell (0, 0x40000000)：CPU物理起始地址cpu_start = 0x00000000_40000000。
- 大小(0, 0x40000000)：size = 1GB。

得到偏移：offset = cpu_start - dma_start = 0x40000000 - 0x80000000 = -0x40000000。

5. 具体换算示例

假设物理地址（paddr）= 0x50000000（此物理页属于区间 [0x40000000, 0x7FFFFFFF] 内）。
则：
dma = paddr - offset
= 0x50000000 - (0x40000000 - 0x80000000)
= 0x50000000 - (-0x40000000)
= 0x50000000 + 0x40000000
= 0x90000000

所以 dma_handle = 0x90000000。

实际就是dma = dma_start + (paddr - cpu_start)。

从 CPU 物理视角看这不是DRAM范围内的地址，但对该PCIe控制器下设备来说它正是应访问的DMA总线地址。硬件inbound窗口会把0x90000000（设备侧）平移回0x50000000对应的DRAM位置。

6. CPU如何访问

如果直接对dma_handle建页表进行访问，很显然得不到正确的结果，甚至会引起系统奔溃。

正确的做法是使用include/linux/dma-mapping.h当中描述的dma接口去访问，以dma_mmap_attrs为例，

```
static inline phys_addr_t translate_dma_to_phys(struct device *dev,
dma_addr_t dma_addr)
{
    const struct bus_dma_region *m;

    for (m = dev->dma_range_map; m->size; m++)
        if (dma_addr >= m->dma_start && dma_addr - m->dma_start < m->size)
            return (phys_addr_t)dma_addr + m->offset;
        return (phys_addr_t)-1;
}
```

dma_mmap_attrs执行过程中会调用translate_dma_to_phys将dma地址转换成真实的CPU物理地址之后，再建立页表，这时候CPU才能正确访问。因此在实际的开发过程需要注意相应的SoC平台是否有dma-ranges的限制。

44. 如何分析内核分配内存失败时打印的log

经常会有如下内存分配失败打印的log：

```
10-14 08:02:10.617   609   609 W android.hardwar: page allocation failure:
order:0, mode:0x140dc6(GFP_HIGHUSER|GFP_DMA32|__GFP_COMP|__GFP_ZERO), nodemask=
(null),cpuset=/,mems_allowed=0
10-14 08:02:10.617   609   609 W         : CPU: 6 UID: 1000 PID: 609 Comm:
android.hardwar Not tainted 6.12.38+ #51
10-14 08:02:10.617   609   609 W Hardware name: Rockchip RK3588 EVB1 LP4 V10
Board (DT)
10-14 08:02:10.617   609   609 W Call trace:
10-14 08:02:10.617   609   609 W         : dump_backtrace+0xec/0x128
10-14 08:02:10.617   609   609 W         : show_stack+0x18/0x28
10-14 08:02:10.617   609   609 W         : dump_stack_lvl+0x40/0xbc
10-14 08:02:10.617   609   609 W         : dump_stack+0x18/0x30
10-14 08:02:10.617   609   609 W         : warn_alloc+0xf4/0x108
10-14 08:02:10.617   609   609 W         : __alloc_pages_slowpath+0x4b8/0x52c
10-14 08:02:10.617   609   609 W         : __alloc_pages_noprof+0x278/0x2e8
10-14 08:02:10.617   609   609 W         : dmabuf_page_pool_alloc+0x58/0x7c
10-14 08:02:10.617   609   609 W         : system_heap_do_allocate+0x284/0x76c
10-14 08:02:10.617   609   609 W         : system_heap_allocate+0x18/0x28
10-14 08:02:10.617   609   609 W         : dma_heap_ioctl+0x24c/0x4a0
10-14 08:02:10.617   609   609 W         : __arm64_sys_ioctl+0xa8/0xe4
10-14 08:02:10.617   609   609 W         : invoke_syscall+0x58/0x10c
10-14 08:02:10.617   609   609 W         : el0_svc_common+0xa8/0xdc
10-14 08:02:10.617   609   609 W         : do_el0_svc+0x1c/0x28
10-14 08:02:10.617   609   609 W         : el0_svc+0x28/0x58
10-14 08:02:10.617   609   609 W         : el0t_64_sync_handler+0x70/0xbc
10-14 08:02:10.617   609   609 W         : el0t_64_sync+0x16c/0x170
10-14 08:02:10.617   609   609 W Mem-Info:
10-14 08:02:10.617   609   609 W active_anon: 0 inactive_anon:112021
isolated_anon:0
10-14 08:02:10.617   609   609 W active_file: 0 inactive_file:228356
isolated_file:0
10-14 08:02:10.617   609   609 W unevictable: 32849 dirty:54 writeback:0
10-14 08:02:10.617   609   609 W slab_reclaimable: 11185
slab_unreclaimable:32007
```

```
10-14 08:02:10.617   609   609 W mapped  : 130536 shmem:1809 pagetables:8626
10-14 08:02:10.617   609   609 W sec_pagetables: 344 bounce:0
10-14 08:02:10.617   609   609 W kernel_misc_reclaimable: 0
10-14 08:02:10.617   609   609 W free    : 636119 free_pcp:579 free_cma:63452
10-14 08:02:10.617   609   609 W         : Node 0 active_anon:0kB
inactive_anon:448084kB active_file:0kB inactive_file:913424kB
unevictable:131396kB isolated(anon):0kB isolated(file):0kB mapped:522144kB
dirty:216kB writeback:0kB shmem:7236kB writeback_tmp:0kB kernel_stack:17792kB
shadow_call_stack:4496kB pagetables:34504kB sec_pagetables:1376kB
all_unreclaimable? no
10-14 08:02:10.617   609   609 W DMA32 free: 257780kB boost:0kB min:5348kB
low:20584kB high:35820kB reserved_highatomic:24576KB free_highatomic:0KB
active_anon:0kB inactive_anon:0kB active_file:0kB inactive_file:0kB
unevictable:0kB writepending:0kB present:3913728kB managed:3809836kB mlocked:0kB
bounce:0kB free_pcp:2316kB local_pcp:2316kB free_cma:253808kB
10-14 08:02:10.617   609   609 W lowmem_reserve[]: 0 4192 4192
10-14 08:02:10.617   609   609 W DMA32   : 5*4kB (MC) 66*8kB (UMC) 3*16kB (MC)
1*32kB (M) 2*64kB (MC) 2*128kB (UM) 1*256kB (C) 1*512kB (U) 2*1024kB (MC)
2*2048kB (UC) 61*4096kB (C) = 257780kB
10-14 08:02:10.617   609   609 W         : 262176 total pagecache pages
10-14 08:02:10.617   609   609 W         : 0 pages in swap cache
10-14 08:02:10.617   609   609 W         : Free swap  = 4049912kB
10-14 08:02:10.617   609   609 W         : Total swap = 4051496kB
10-14 08:02:10.617   609   609 W         : 2092544 pages RAM
10-14 08:02:10.617   609   609 W         : 0 pages HighMem/MovableOnly
10-14 08:02:10.617   609   609 W         : 66793 pages reserved
10-14 08:02:10.617   609   609 W         : 65536 pages cma reserved
```

从以上的log可以得知以下重点信息：

1. 分配内存大小为2^0个page，即4KB。
2. 由用户进程android.hardware通过system heap分配。
3. 分配的标志位(`GFP_HIGHUSER|GFP_DMA32|__GFP_COMP|__GFP_ZERO`)
    - GFP_HIGHUSER

      不同于GFP_HIGHUSER_MOVABLE，GFP_HIGHUSER分配的内存是不能借用CMA的，因为dmabuf的内存一般用于外设，意味着page不可移动，所以不允许借用CMA是合理的。
    - GFP_DMA32

      只能从低32位物理内存空间申请，即只能从DMA32 ZONE申请page，可能设备只有访问低32位物理地址的能力。
    - __GFP_COMP

      允许申请复合页，需要额外设置page的标志位。
    - __GFP_ZERO

      申请到的page需要清零。
4. DMA32 ZONE汇总信息

```
DMA32 free: 257780kB boost:0kB min:5348kB low:20584kB high:35820kB
reserved_highatomic:24576KB free_highatomic:0KB active_anon:0kB
inactive_anon:0kB active_file:0kB inactive_file:0kB unevictable:0kB
writepending:0kB present:3913728kB managed:3809836kB mlocked:0kB bounce:0kB
free_pcp:2316kB local_pcp:2316kB free_cma:253808kB
```

    - free: 该 zone 当前空闲页总量（包含 CMA 的空闲页）。

- boost: 水位提升量（各种原因分配失败需要临时抬高水位线的额外值）。
- min/low/high: 三条水位线。空闲量低于 low 时 kswapd 会回收；低于 min 可能触发直接回收/失败；high 是回收到的目标。
- reserved_highatomic: 为“原子分配（GFP_ATOMIC）”预留的页，普通分配不可用。
- free_highatomic: 当前仍处于 highatomic 预留中的可用页。
- active_anon/inactive_anon: 匿名内存（如堆、栈）在活跃/非活跃 LRU 列表上的page数量。
- active_file/inactive_file: 文件页（页缓存）在活跃/非活跃 LRU 列表上的page数量。
- unevictable: 不可回收页（例如被 mlock 锁定、巨页等）。
- writepending: 正在写回/脏页数量。
- present: 该 zone 覆盖的物理内存总量。
- managed: 该 zone 由伙伴系统实际管理的可分配页（present 扣除洞、保留等）。
- mlocked: 被 mlock 锁定的页（不可回收）。
- bounce: bounce buffer 相关页（老设备/驱动用，常见为 0）。
- free_pcp/local_pcp: 每 CPU 的本地空闲页缓存量；local_pcp 是当前 CPU 的。
- free_cma: CMA区域内的空闲页。计入 free，但通常只对“可移动”分配可用，对不可移动分配（Unmovable）不可用。

5. DMA32 ZONE可用内存各个order的链表信息

```
DMA32    : 5*4kB (MC) 66*8kB (UMC) 3*16kB (MC) 1*32kB (M) 2*64kB (MC) 2*128kB
(UM) 1*256kB (C) 1*512kB (U) 2*1024kB (MC) 2*2048kB (UC) 61*4096kB (C) =
257780kB
```

ZONE的空闲页按以下图示组织：

```
Zone
└─ free_area[0]  (order=0, 每块=1页)
   ├─ free_list[MIGRATE_MOVABLE]      : [p]-[p]-[p]--->
   ├─ free_list[MIGRATE_UNMOVABLE]    : [p]-[p]--->
   ├─ free_list[MIGRATE_RECLAIMABLE]  : [p]--->
   ├─ free_list[MIGRATE_HIGHATOMIC]   : [p]-[p]-[p]-[p]--->
   ├─ free_list[MIGRATE_CMA]          : [p]--->
   └─ free_list[MIGRATE_ISOLATE]      : [p]--->    (通常隔离用，常为空)

└─ free_area[1]  (order=1, 每块=2页)
   ├─ free_list[MIGRATE_MOVABLE]      : [pp]-[pp]--->
   ├─ free_list[MIGRATE_UNMOVABLE]    : [pp]--->
   ├─ free_list[MIGRATE_RECLAIMABLE]  : --->
   ├─ free_list[MIGRATE_HIGHATOMIC]   : [pp]--->
   ├─ free_list[MIGRATE_CMA]          : --->
   └─ free_list[MIGRATE_ISOLATE]      : --->

└─ free_area[2]  (order=2, 每块=4页)
   ├─ free_list[MIGRATE_MOVABLE]      : [pppp]--->
   ├─ free_list[...]
   └─ ...

... 一直到 free_area[MAX_ORDER-1]
```

- 方括号里的 p 代表页；[pp] 代表 2 页连续块；[pppp] 代表 4 页连续块，以此类推。
- 同一 order 的不同 migratetype 是不同的双向链表（list_head）。
- 分配与释放简单流程
    - 分配：

1. 选定目标 migratetype（由 alloc_context/mask 推导，如 GFP_MOVABLE → MOVABLE；GFP_ATOMIC → 可能带 ALLOC_HIGHATOMIC）。
2. 在 free_area[order].free_list[target_mt] 查找；
3. 若无，则按回退策略在其它 mt 查找；或向上找更高 order 并层层拆分（split）。

- 释放：

1. 与伙伴合并（coalesce）直至无法再合并；
2. 根据该页所在 pageblock 的 migratetype，放回对应 order 的 free_list（小阶可能先进 per-CPU PCP，再批量灌回 zone）

```
static void show_migration_types(unsigned char type)
{
static const char types[MIGRATE_TYPES] = {
    [MIGRATE_UNMOVABLE] = 'U',
    [MIGRATE_MOVABLE]   = 'M',
    [MIGRATE_RECLAIMABLE]   = 'E',
    [MIGRATE_HIGHATOMIC]    = 'H',
#ifdef CONFIG_CMA
    [MIGRATE_CMA]       = 'C',
#endif
#ifdef CONFIG_MEMORY_ISOLATION
    [MIGRATE_ISOLATE]   = 'I',
#endif
};
char tmp[MIGRATE_TYPES + 1];
char *p = tmp;
int i;

for (i = 0; i < MIGRATE_TYPES; i++) {
    if (type & (1 << i))
        *p++ = types[i];
}

*p = '\0';
printk(KERN_CONT "(%s) ", tmp);
}
```

内核代码mm/show_mem.c里面说明了字符'U','M','E','H','C'的含义。

linux内核的order取值范围一般是[0 10]，随机分析几个：

- 5*4kB (MC)

  表示DMA32 ZONE里，order为0的空闲页由MIGRATE_MOVABLE/MIGRATE_CMA组成。

- 66*8kB (UMC)

  表示DMA32 ZONE里，order为1的空闲页由MIGRATE_UNMOVABLE/MIGRATE_MOVABLE/MIGRATE_CMA组成。

- 61*4096kB (C)

  表示DMA32 ZONE里，order为10的空闲页都是MIGRATE_CMA。

6. 有了以上的知识，最后回到问题本身，为什么一个页都申请不到？

   从链表信息可以看出DMA32 ZONE的空闲内存有257780kB，但实际上绝大多数都是CMA，因为分配标志位不允许借用CMA，所以在计算water mark的时候需要减去CMA部分，再结合min:5348kB信息，water mark远低于min水线，从而导致从DMA32 ZONE分配失败。

45. 如何获取PAGE的OWNER信息

有些时候我们需要知道某个PAGE被哪个进程占用，内核提供了记录page owner信息的功能，打开COFIG_PAGE_OWNER并在内核command line添加`page_owner=on`即可，通过以下命令可以获取到page owner的信息：

```
cat /sys/kernel/debug/page_owner

Page allocated via order 0, mask
0x8152c4a(GFP_NOFS|__GFP_HIGHMEM|__GFP_NOWARN|__GFP_NORETRY|__GFP_COMP|__GFP_HAR
DWALL|__GFP_MOVABLE|__GFP_CMA), pid 1635, tgid 1635 (droid.launcher3), ts
11384184706 ns, free_ts 11022307749 ns
PFN 3b004 type Movable Block 236 type CMA Flags
0x1000000000020014(uptodate|lru|mappedtodisk|zone=0)
 prep_new_page+0x164/0x16c
 get_page_from_freelist+0x1760/0x17bc
 __alloc_pages+0x140/0x38c
 __folio_alloc+0x14/0x20
 page_cache_ra_unbounded+0x100/0x234
 page_cache_ra_order+0x40/0x4c
 do_sync_mmap_readahead+0x1c8/0x230
 filemap_fault+0x1ac/0x588
 __do_fault+0xa8/0xf8
 handle_mm_fault+0x3d4/0xec0
 do_page_fault+0x1c0/0x49c
 do_translation_fault+0x38/0x50
 do_mem_abort+0x40/0xec
 el0_da+0x3c/0x8c
 el0t_64_sync_handler+0x98/0xb4
 el0t_64_sync+0x164/0x168
Charged to memcg /
```

以上是进程`droid.launcher3`因为访问文件陷入内核获得物理页0x3b004的堆栈信息，以下详细分析：

1. `GFP_NOFS|__GFP_HIGHMEM|__GFP_NOWARN|__GFP_NORETRY|__GFP_COMP|__GFP_HARDWALL|__GFP_MOVABLE|__GFP_CMA`

进程申请内存使用gfp_flag，重点是`__GFP_MOVABLE`，表示申请可移动的页，因此允许从CMA获取，最后的标志位带上了`__GFP_CMA`。

2. `PFN 3b004 type Movable Block 236 type CMA`

PFN 0x3b004表示获取到物理页帧号，type Movable表示该页的migrate_type是Movable。Block 236 type CMA表示从第236个pageblock获取到的物理页，该pageblock的type是CMA。以4M pageblock为例，Block 236涵盖的pfn范围为236 × 4 × 1024 × 1024 / 4096 至 237 × 4 × 1024 × 1024 / 4096，即[0x3b000 0x3b400], 0x3b004刚好落在这个pageblock。

3. `uptodate|lru|mappedtodisk|zone=0`

从堆栈可以看出进程申请内存用于文件缓存即文件页page cache，所以这几个flag都跟文件相关：

uptodate表示页面内容是最新的，不需要再做I/O操作。

lru表示页面在某个类型的lru链表里面，lru链表里面管理的页面可以被用于回收或者swap。因此低内存时候优先从lru链表回收page。

mappedtodisk表示这个是文件页而不是匿名页。

zone=0表示该PFN所属的zone编号，一般0表示ZONE_DMA32或者ZONE_DMA。

从以上log就可以观察到CMA page被谁借用，有助于分析问题。但是`/sys/kernel/debug/page_owner`节点会将所有page信息都打印出来，信息量非常大，有时候我们只想观察特定的page，内核提供了接口函数dump_page_owner(page)，只会dump特定page的owner信息。如下：

```
isolate_migratepages_block,1049 abort by LRU, low_pfn:5b3d8
page_owner tracks the page as freed
page last allocated via order 0, migratetype Movable, gfp_mask
0x8140cca(GFP_HIGHUSER_MOVABLE|__GFP_COMP|__GFP_CMA), pid 2183, tgid 2183
(putmethod.latin), ts 56068406577, free_ts 63927064672
 prep_new_page+0x164/0x16c
 get_page_from_freelist+0x1760/0x17bc
 __alloc_pages+0x140/0x38c
 __folio_alloc+0x14/0x20
 do_swap_page+0x614/0xaf8
 handle_mm_fault+0x498/0xec0
 do_page_fault+0x1c0/0x49c
 do_translation_fault+0x38/0x50
 do_mem_abort+0x40/0xec
 el0_da+0x3c/0x8c
 el0t_64_sync_handler+0x98/0xb4
 el0t_64_sync+0x164/0x168
page last free stack trace:
 free_unref_page_prepare+0x3d8/0x3ec
 free_unref_page_list+0x8c/0x384
 shrink_folio_list+0xd84/0xe54
 evict_folios+0x161c/0x1984
 try_to_shrink_lruvec+0x27c/0x344
 shrink_one+0xb0/0x1f8
 shrink_node+0xac4/0xe50
 balance_pgdat+0x4f4/0x8d0
 kswapd+0x2b0/0x45c
 kthread+0xec/0x1b8
 ret_from_fork+0x10/0x20
```

以上log包含了PFN 0x5b3db的owner信息，包括最近一次的分配和释放路径，当前处于freed状态。上面的log是在分析某个page隔离失败时候添加dump_page_owner产生的。因此，如果想要跟踪特定的page，可以尝试在合适的代码路径上添加dump_page_owner函数。

46. 如何获取page pinner信息

page pinner用来检测page的ref_count和map_count这两个计数值的变化，GUP返回的page会将ref_count计数+1以此锁住page，防止page被回收/迁移/整理。打开内核page pinner功能需要打开CONFIG_PAGE_PINNER，并且内核命令行需要配置`page_pinner=on`。

```
cat /sys/kernel/debug/page_pinner/buffer
```

以上命令会打印page pinner的信息，如下

```
At least, pinned for 101359 us
```

```
PFN 0x76073 Block 472 count 2 mapcount 1 mapping 0x0 Flags
0x20000000000a0016(0x20000000000a0016(referenced|uptodate|lru|mappedtodisk|swapb
acked|zone=0))
 isolate_migratepages_block+0x7ec/0x11a8
 isolate_migratepages_range+0xb4/0xf0
 __alloc_contig_migrate_range+0xdc/0x31c
 alloc_contig_range+0x120/0x364
 __cma_alloc+0x250/0x600
 cma_alloc+0x20/0x2c
 rk_secure_heap_allocate+0x2b4/0x464
 dma_heap_ioctl+0x3b4/0x47c
 __arm64_sys_ioctl+0x90/0xc8
 invoke_syscall+0x40/0x104
 el0_svc_common+0xbc/0x168
 do_el0_svc+0x1c/0x28
 el0_svc+0x1c/0x68
 el0t_64_sync_handler+0x68/0xb4
 el0t_64_sync+0x164/0x168

Failure detected at [   65.431710]
PFN 0x76073 Block 472 count 2 mapcount 1 mapping 0x0 Flags
0x20000000000a0016(0x20000000000a0016(referenced|uptodate|lru|mappedtodisk|swapb
acked|zone=0))
 test_pages_isolated+0x384/0x398
 alloc_contig_range+0x2a4/0x364
 __cma_alloc+0x250/0x600
 cma_alloc+0x20/0x2c
 rk_secure_heap_allocate+0x2b4/0x464
 dma_heap_ioctl+0x3b4/0x47c
 __arm64_sys_ioctl+0x90/0xc8
 invoke_syscall+0x40/0x104
 el0_svc_common+0xbc/0x168
 do_el0_svc+0x1c/0x28
 el0_svc+0x1c/0x68
 el0t_64_sync_handler+0x68/0xb4
 el0t_64_sync+0x164/0x168
```

上面的log是执行cma_alloc失败之后获取到的page pinner的信息，可以看出，count的值比mapcount大，说明该0x76073这个page被"锁住了"，因为cma_alloc的要获取的pfn范围包含了0x76073，导致cma_alloc失败。这时候再结合page owner的信息，就有可能知道page被谁"锁住"。如下

```
[   67.517790][ T2363] page_owner tracks the page as allocated
[   67.517804][ T2363] page[0x76073] last allocated via order 0, migratetype
Movable, gfp_mask
0x8140dca(GFP_HIGHUSER_MOVABLE|__GFP_COMP|__GFP_ZERO|__GFP_CMA), pid 2353, tgid
2353 (user_test), ts 28571615265, free_ts 28564360348
[   67.517836][ T2363]  prep_new_page+0x164/0x16c
[   67.517866][ T2363]  get_page_from_freelist+0x1504/0x1560
[   67.517891][ T2363]  __alloc_pages+0x124/0x2bc
[   67.517914][ T2363]  __folio_alloc+0x14/0x20
[   67.517938][ T2363]  alloc_zeroed_user_highpage_movable+0x28/0x34
[   67.517960][ T2363]  handle_mm_fault+0x70c/0xea0
[   67.517980][ T2363]  do_page_fault+0x258/0x49c
[   67.517999][ T2363]  do_translation_fault+0x38/0x50
[   67.518019][ T2363]  do_mem_abort+0x40/0xec
[   67.518037][ T2363]  el0_da+0x3c/0x8c
[   67.518063][ T2363]  el0t_64_sync_handler+0x98/0xb4
```

```
[   67.518089][ T2363]  el0t_64_sync+0x164/0x168
[   67.518106][ T2363] page last free stack trace:
[   67.518119][ T2363]  free_unref_page_prepare+0x3d8/0x3ec
[   67.518143][ T2363]  free_unref_page_list+0x8c/0x384
[   67.518165][ T2363]  release_pages+0x3d8/0x414
[   67.518185][ T2363]  free_pages_and_swap_cache+0x4c/0x60
[   67.518203][ T2363]  tlb_finish_mmu+0xa0/0x12c
[   67.518229][ T2363]  exit_mmap+0x180/0x39c
[   67.518254][ T2363]  __mmput+0x28/0x134
[   67.518272][ T2363]  mmput+0x28/0x5c
[   67.518288][ T2363]  exec_mmap+0x20c/0x2ac
[   67.518306][ T2363]  begin_new_exec+0x398/0x59c
[   67.518325][ T2363]  load_elf_binary+0x29c/0xa58
[   67.518350][ T2363]  bprm_execve+0x2e4/0x5a4
[   67.518368][ T2363]  do_execveat_common+0x258/0x2c4
[   67.518388][ T2363]  __arm64_sys_execve+0x40/0x54
[   67.518408][ T2363]  invoke_syscall+0x40/0x104
[   67.518431][ T2363]  el0_svc_common+0xbc/0x168
```

从page owner信息得知0x76073这个page是被user_test应用占用，user_test正是用来测试GUP的应用程序。user_test将malloc获取到用户态虚拟地址给到内核驱动执行GUP之后便返回，从而导致page被锁住。

page pinner 主要用于检测 page 的引用状态。在实际编码中，可以在合适的内核代码路径中插入 page pinner 标记，用来观测和校验 page 的引用情况。内核已经提供了 page_pinner_failure_detect 函数，可在检测到异常引用（如引用计数与映射计数不一致）时触发记录，帮助定位问题。

47. scatterlist 使用注意事项(IOMMU/DMABUF场景)

47.1 背景与基本概念

```
struct scatterlist {
    unsigned long   page_link;
    unsigned int    offset;
    unsigned int    length;
    dma_addr_t  dma_address;
#ifdef CONFIG_NEED_SG_DMA_LENGTH
    unsigned int    dma_length;
#endif
#ifdef CONFIG_PCI_P2PDMA
    unsigned int    dma_flags;
#endif
};
```

- `page_link`：编码后的指针/链表信息，用于定位该段对应的 `struct page`，并在部分场景下组织 SG 链(chain）。

  在 `scatterlist` 中，单个 `struct scatterlist` 仅表示一段连续地址空间。为了描述多段非连续地址空间，需要将多个 `scatterlist` 组织成一个表/链。为减少额外字段开销，内核复用了 `page_link` 的低位来编码链表关系与结束标记：

  - 当 bit0 = 1 时，表示该 `scatterlist` 为链表节点（chain），此时 `page_link`（去掉低位标志后）保存 下一个 **`scatterlist`** 的地址。

- 当 bit1 = 1 时，表示该 `scatterlist` 为 **最后一个条目（end / last）**。

因此，遍历SG时不能简单地把 `page_link` 当作纯page指针使用，需要通过内核提供的接口（如 `sg_next()`、`sg_is_last()`、`sg_is_chain()` 等）来正确解析和遍历。

- `offset`：该段在 page 内的偏移(byte)。
- `length`：该段连续内存的长度(byte)，反映原始内存片段长度。
- `dma_address`：DMA 映射后设备访问的地址(可能是物理地址，也可能是 IOVA，取决于是否启用IOMMU/是否走DMA mapping)。
- `dma_length`：在 `CONFIG_NEED_SG_DMA_LENGTH` 使能时存在，用于记录 DMA 映射后的长度信息(注意：它的语义在某些映射/合并场景下与 `length` 不同)。

  相关代码示例：

```
#ifdef CONFIG_NEED_SG_DMA_LENGTH
#define sg_dma_len(sg)       ((sg)->dma_length)
#else
#define sg_dma_len(sg)       ((sg)->length)
#endif

static inline struct page *sg_page(struct scatterlist *sg)
{
    return (struct page *)((sg)->page_link & ~SG_PAGE_LINK_MASK);
}

static inline dma_addr_t sg_phys(struct scatterlist *sg)
{
    return page_to_phys(sg_page(sg)) + sg->offset;
}
```

  通常使用 `sg_table` 表示一个 SG 列表：

```
struct sg_table {
    struct scatterlist *sgl;    /* the list */
    unsigned int nents;     /* number of mapped entries */
    unsigned int orig_nents;    /* original size of list */
};
```

  - `sgl`：指向第一个 `scatterlist` 条目
  - `orig_nents`：原始条目数(未进行 DMA 映射合并前)
  - `nents`：DMA 映射后的条目数(可能发生段合并/拆分，取决于DMA/IOMMU映射结果)

47.2 常见问题与正确用法(重点：启用IOMMU + CONFIG_NEED_SG_DMA_LENGTH)

1. 用sg_dma_len()遍历做“物理维度操作”会得到异常长度

   现象

   很多驱动/模块在遍历SG时，会读取物理地址与长度做额外操作(例如 cache flush、内存校验、统计等)。实践中可能出现：

   - sg_phys(sg)返回的物理首地址看起来正确
   - 但sg_dma_len(sg)返回的长度“异常”(例如第一个条目长度变成所有条目的总和，其它条目长度为0)

原因

sg_dma_len(sg)的语义是DMA映射维度的长度。在IOMMU/DMAMAP场景下，DMA映射逻辑可能将多个物理段合并为更少的IOVA段，因此dma_length/dma_address体现的是映射后的视角，而非原始物理段视角。

第一个SG 的dma_length = 合并后的总长度。

其它SG的dma_length/dma_address置0这会导致按sg_dma_len() 去做“逐物理段”操作时逻辑错误。

正确做法

如果你的操作是“按原始物理段粒度”进行(例如按page/物理段刷cache、做物理段遍历统计)，应使用：

长度：sg->length

物理地址：sg_phys(sg) (见第3点DMABUF 注意事项)。

换句话说：

DMA 相关(设备访问)：使用 sg_dma_address()/sg_dma_len()

原始内存相关(CPU/物理段处理)：使用sg_phys()/sg->length

2. 错用sg_table->nents遍历原始SG，导致越界/崩溃

现象

驱动中常见写法：用sgtable->nents作为遍历次数去解析sgl，做cache flush等操作。启用IOMMU后可能出现内核崩溃、非法访问、遍历行为与预期不一致。

原因

`nents` 表示DMA映射后的条目数。DMA/IOMMU映射可能将多个物理段合并成更少的IOVA段，因此常见情况是：

- `orig_nents = 5`
- DMA 映射后 `nents = 1` 此时若仍按 `nents` 去做原始SG的逐段处理，就会漏处理，引发不一致行为。

正确做法

- 需要按“原始条目”处理SG(例如物理段遍历、原始 cache 维护)：使用 `orig_nents` 或者使用内核提供的遍历宏 `for_each_sgtable_sg()`。
- DMA cache 同步、映射/解映射等 DMA 相关操作：优先使用内核封装接口，避免自行解析：

推荐接口：

- `dma_sync_sgtable_for_cpu()`
- `dma_sync_sgtable_for_device()`
- `dma_map_sgtable()`
- `dma_unmap_sgtable()`

这些接口会正确处理 `nents/orig_nents`、IOMMU 映射合并、cache 维护与体系结构差异。

3. DMABUF + CONFIG_DMABUF_DEBUG：直接解析sg_page()/sg_phys()可能得到异常地址(用于抓违规用法)

现象

当某个 `scatterlist` 来自DMABUF(消费者 importer侧拿到的sgt)，并且打开了 `CONFIG_DMABUF_DEBUG`，可能观察到：

- `sg_phys(sg)` 返回的物理地址异常
- 若未做检查就继续刷cache/访问该物理地址，可能导致系统崩溃

原因

DMABUF框架在debug模式下会“污染/混淆”SG 表中的page指针信息，以捕捉importer侧滥用底层 `struct page` 的行为：

```
static void mangle_sg_table(struct sg_table *sg_table)
{
#ifdef CONFIG_DMABUF_DEBUG
    int i;
    struct scatterlist *sg;

    /* To catch abuse of the underlying struct page by importers mix
     * up the bits, but take care to preserve the low SG_ bits to
     * not corrupt the sgt. The mixing is undone in __unmap_dma_buf
     * before passing the sgt back to the exporter. */
    for_each_sgtable_sg(sg_table, sg, i)
        sg->page_link ^= ~0xffUL;
#endif
}
```

注释含义很明确：**importer不应直接依赖/解析 sg 内的 page 信息**；debug模式通过混淆来暴露这类不合规用法，混淆会在 `__unmap_dma_buf` 前被恢复。

正确做法

- 在 DMABUF 场景下，importer侧应通过DMABUF提供的标准接口完成映射/同步，而不是直接去解析 `sg_page()` / `sg_phys()` 得到物理地址进行操作。
- 建议在开发/联调阶段开启 `CONFIG_DMABUF_DEBUG`，用来尽早发现不合规的DMABUF使用方式。

48. CMA_ALLOC返回失败

有些经过内核配置优化的版本在启动过程中可能会出现如下报错：

```
[    1.362183] cma: cma_alloc: cma: alloc failed, req-size: 512 pages, ret: -38
[    1.362310] DMA: preallocated 2048 KiB GFP_KERNEL pool for atomic allocations
[    1.362398] cma: cma_alloc: cma: alloc failed, req-size: 512 pages, ret: -38
[    1.362501] DMA: preallocated 2048 KiB GFP_KERNEL|GFP_DMA pool for atomic
allocations
[    1.362517] cma: cma_alloc: cma: alloc failed, req-size: 512 pages, ret: -38
[    1.362618] DMA: preallocated 2048 KiB GFP_KERNEL|GFP_DMA32 pool for atomic
allocations
```

需要注意的是，即使从启动日志看CMA的初始化过程正常，运行时仍可能出现 `cma_alloc` 分配失败的情况。该问题较大概率与`CONFIG_MIGRATION`被关闭有关。

CMA的正常工作依赖“内存迁移（memory migration）”能力：当CMA需要分配一段连续物理内存时，内核通常需要通过迁移页面的方式整理出满足要求的连续区域。如果关闭了`CONFIG_MIGRATION`，意味着内存迁移功能不可用，CMA在运行时就无法完成页面整理，从而导致`cma_alloc`异常失败。

在一些对实时性要求较高的场景中，可能会出于提高系统实时性的目的而关闭`CONFIG_MIGRATION`。因此，是否开启`CONFIG_MIGRATION`需要结合具体业务需求权衡配置：若系统依赖CMA为设备提供连续内存，通常应保持 `CONFIG_MIGRATION`开启；若实时性优先且业务不依赖CMA，则可评估关闭带来的影响。

49. ARM32下CMA起始地址未2M对齐导致内核启动卡死

49.1 现象描述

在ARM32架构下，如果DTS中配置的CMA内存区域落在内核线性映射区（即物理地址小于全局变量 `arm_lowmem_limit`），并且起始地址**没有按照2MB对齐**，会导致正常的内核页表被破坏，进而引发内核在启动过程中卡死。

49.2 原因分析

内核在初始化DMA内存映射时会调用 `dma_contiguous_remap()` 函数。该函数为了确保TLB（Translation Lookaside Buffer）中没有冲突的旧条目，会清除该区域之前的低端内存映射：

```
void __init dma_contiguous_remap(void)
{
    // ...
    /*
     * Clear previous low-memory mapping to ensure that the
     * TLB does not see any conflicting entries...
     */
    for (addr = __phys_to_virt(start); addr < __phys_to_virt(end);
         addr += PMD_SIZE)
        pmd_clear(pmd_off_k(addr));

    // ...
}
```

这里的问题在于，`pmd_clear` 会强制按照 `PMD_SIZE`（在ARM32常规配置下为2MB）对齐来清除段映射页表项。如果CMA的起始地址（`start`）本身不是2MB对齐的，`pmd_clear` 操作就会顺带将包含该地址的整个2MB段清零，这往往会误删相邻的、正处于使用状态的正常内核映射空间，从而导致内核挂死。

为什么会出现不对齐的情况？
内核在启动前期会解析 `reserved-memory` 的子节点，并根据节点的配置来分配预留内存。预留内存的地址对齐受两个主要配置影响：

1. 节点本身的 `alignment` 参数。
2. 内核宏 `CONFIG_ARCH_FORCE_MAX_ORDER`。

具体代码执行位置在 `drivers/of/of_reserved_mem.c` 的 `__reserved_mem_alloc_size` 函数中。有些内核会将CONFIG_ARCH_FORCE_MAX_ORDER配置为9，导致CMA的预留地址对齐到1MB。如果在分配时恰好落在1MB对齐但非2MB对齐的边界，加上该地址位于线性映射区内，就会触发上述崩溃。

> 补充说明：有一些环境由于开启了 `CONFIG_CMA_INACTIVE`，或者CMA预留的地址碰巧不在内核线性映射区域内（即大于 `arm_lowmem_limit`），从而幸运地规避了这个问题。

49.3 解决方法

最简单、直接的规避方案是在DTS配置中，明确指定CMA区域必须按照2MB对齐。

代码修改示例：

在 `reserved-memory` 的 `linux,cma` 节点中添加 `alignment = <0x00200000>;`：

```
    cma: linux,cma {
        compatible = "shared-dma-pool";
        reusable;
        size = <0x00000000>; /* Size由具体项目决定 */
        linux,cma-default;
        alignment = <0x00200000>; /* 新增：强制按照2MB对齐 */
    };
```